

Major Buckets of Non-Human Activity in 2022 R/Place

Rectangular/area pixel placements

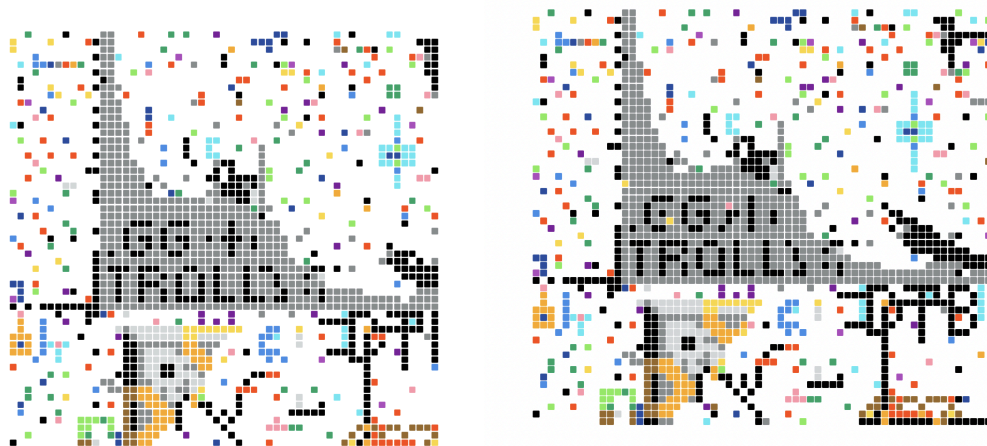
In the `r_place` dataset, there are 19 rows that have 4 numbers provided as the action's coordinate (x, y, x, y) instead of the regular two (x, y). These irregular actions represent an area, or rectangle, of simultaneous pixel placements rather than the singular pixels placed by regular users. rectangular placements are likely efforts by privileged users to moderate the `r_place` canvas, covering images that go against Reddit policy.

The 19 rectangular pixel placements, are:

```
('1372,1472,1406,1497',)
('1375,1355,1424,1399',)
('1349,1718,1424,1752',)
('1373,1400,1419,1436',)
('1371,1438,1418,1472',)
('298,1805,329,1839',)
('257,1736,296,1780',)
('271,1835,296,1859',)
('298,1770,334,1803',)
('297,1750,364,1813',)
('251,1805,296,1812',)
('551,1311,562,1342',)
('547,1330,550,1342',)
('44,1652,165,1899',)
('51,1691,154,1807',)
('23,1523,172,1792',)
('871,546,878,550',)
('862,540,868,544',)
('862,540,873,545',)
```

We can explore the purpose of these rectangular placements by viewing the canvas state right before the rectangle was placed. We can also confirm our suspicions that the rectangles are placed for moderation purposes.

The python file, `show_rect_placement.py`, plots local parts of the `r/place` canvas where rectangles were placed the moment before the rectangle was placed. I include a 25 pixel border in each direction to give more context to the image. Unfortunately most of the images are pretty graphic so I'll spare you from those, but here are a couple that were covered by rectangles for example.



Interestingly, when we run the program we can see multiple matching canvas plots. In the case above, even the stray pixels match, so the dataset probably has duplicates of the rectangle placement row. However in other duplicate images, it looks like users were recreating their work and were getting moderated a second time.

Abnormally rapid placements

In the r/place event, users have a 5-second cooldown between pixel placements. If this rule were enforced 100% of the time, we would expect all placements by any given user in our dataset to have at least a 5 minute gap.

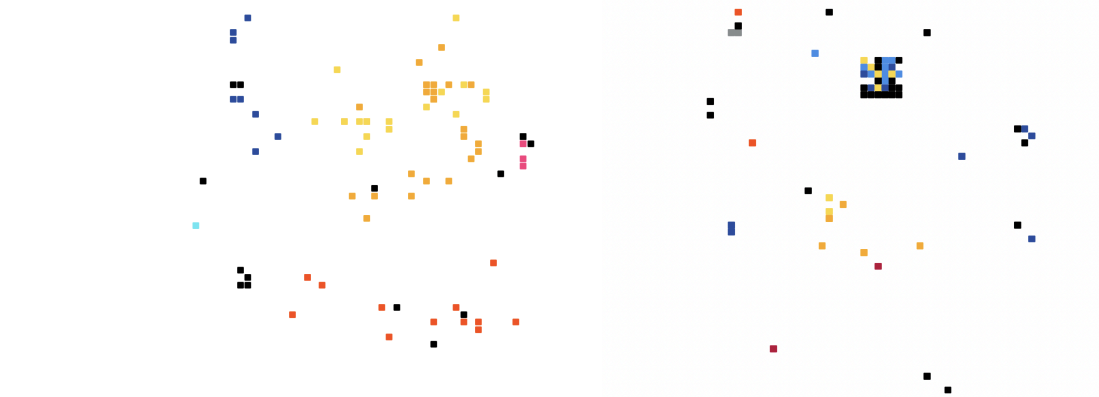
I'll call these abnormally fast consecutive placements by a given user 'fast placements'. Querying the dataset, we can see there are 10432 fast placements with less than a 4.5 minute gap (giving the rule some buffer). These are placements where either users were able to skirt the 5 minute cooldown, reddit's system accidentally let users place too early, or privileged users without a cooldown placed pixels rapidly.

Among fast placers, some users made many more fast placements than others. The top 10 fast placement counts among users were:

67, 48, 46, 33, 27, 27, 26, 24, 23, 22

So fast placements were not one-off occurrences.

Many of these fast-placing accounts only place one or two colors at a set of one or two coordinates, indicating that they're probably bots. These are plotted pixels of some of the more interesting rapid placements:



Suspiciously Timed Placements.

There isn't much information available to us to be able to identify bot users. We could expect bots to place pixels at a very consistent interval. `Show_fast_placements.py` will identify the top 20 fast-placing users, and plot their pixels on the canvas near the median of their activity.

If we query the top-10 users with the most consistent timing, we see:

Users with most consistent placement intervals:

User ID: 3976986, Std Dev: 0.16823851333275977, Avg: 329.96424999999994, Count: 20
User ID: 7625487, Std Dev: 0.20129867607425594, Avg: 300.73009677419356, Count: 31
User ID: 3214864, Std Dev: 0.28048089115486746, Avg: 303.04530303030293, Count: 33
User ID: 2087183, Std Dev: 0.29367678142484166, Avg: 301.0783636363636, Count: 22
User ID: 10361350, Std Dev: 0.2961384701562578, Avg: 301.0552272727272, Count: 22
User ID: 8562374, Std Dev: 0.30392799163861434, Avg: 331.0200909090909, Count: 44
User ID: 4348211, Std Dev: 0.3237133379361596, Avg: 300.9289880952381, Count: 84
User ID: 3366412, Std Dev: 0.33285873011013245, Avg: 303.62375000000001, Count: 20
User ID: 7674708, Std Dev: 0.3540064965040861, Avg: 301.2831363636364, Count: 22
User ID: 9738669, Std Dev: 0.35557584309573387, Avg: 331.03832608695643, Count: 46

It's worth noting, since the cooldown between placements is 5 minutes, any users near an average of 300 seconds on this list were likely real people placing their pixels right when their cooldown ended. If we filter, only selecting users with an average gap greater than 305 seconds, we get a much different list:

Users with most consistent placement intervals with over 305 second gaps:

User ID: 3976986, Std Dev: 0.16823851333275977, Avg: 329.96424999999994, Count: 20
User ID: 8562374, Std Dev: 0.30392799163861434, Avg: 331.0200909090909, Count: 44
User ID: 9738669, Std Dev: 0.35557584309573387, Avg: 331.03832608695643, Count: 46
User ID: 6297841, Std Dev: 0.4021570697030207, Avg: 305.05739393939393, Count: 33
User ID: 6510586, Std Dev: 0.4261931964410706, Avg: 1201.7375454545454, Count: 22
User ID: 7230824, Std Dev: 0.4349350416742696, Avg: 712.4699047619049, Count: 21
User ID: 5083399, Std Dev: 0.4507460056841384, Avg: 306.2734642857143, Count: 28
User ID: 9947727, Std Dev: 0.4667279740732204, Avg: 1201.2867058823529, Count: 34
User ID: 6145218, Std Dev: 0.46787881843974427, Avg: 1201.7012727272725, Count: 22
User ID: 2899680, Std Dev: 0.46927060700346834, Avg: 1201.6739545454548, Count: 22

If we compare these to a random set of users from the pool, it becomes pretty apparent that the ones shown above are bots:

Random set of users' placement intervals:

User ID: 4062220, Std Dev: 13484.839801798262, Avg: 4746.041027777778, Count: 36
User ID: 9009501, Std Dev: 18089.644174123, Avg: 7854.448741935484, Count: 31
User ID: 6702232, Std Dev: 10831.130593195538, Avg: 2324.4629999999997, Count: 69
User ID: 1217807, Std Dev: 4940.873456504814, Avg: 1515.3365483870969, Count: 62
User ID: 5083113, Std Dev: 13315.587441255862, Avg: 4455.186272727271, Count: 22
User ID: 7225247, Std Dev: 2633.6407037111985, Avg: 1001.1520535714286, Count: 56
User ID: 8276244, Std Dev: 15702.714260563427, Avg: 7423.186032258064, Count: 31
User ID: 2644826, Std Dev: 16735.833354672133, Avg: 9700.922375, Count: 24
User ID: 5482211, Std Dev: 13729.832986665824, Avg: 6489.110325, Count: 40
User ID: 4466944, Std Dev: 9501.361861772328, Avg: 3257.4957341772147, Count: 79

The code to find these consistently-timed users can be found in `find_bots.py`